

AIPL / ABCL^c+ 言語ガイド

— 型システムの改修を反映した現状の仕様と使い方 —

ABCL^c+ / AIOS 処理系 (yaskodama/abclcp, branch make-src-base)

2026 年 07 月 30 日 07:22 JST 版

対象コミット: 04892cf (branch make-src-base)

概要

本書は AIPL (ABCL^c+) の**現状**の言語ガイドである。第 I 部で今回の改修による変更点と移行方法をまとめ、第 II 部で言語仕様、第 III 部でサンプルプログラムをソースコードと実行結果つきで説明し、第 IV 部でビルドと実行の手順を示す。

今回の改修の中心は、メソッドの戻り値型を `reply` から推論できるようにし、さらに省略可能な戻り値型注釈 `method m(x) : T` を導入したことである。これにより `now / future` の結果が `any` ではなく具体的な型を持つようになった。あわせて文字列連結を `+` から `++` へ分離し、曖昧なオーバーロードをエラーに格上げし、メソッド本体の仮引数をシグネチャに結線した。設計の根拠と測定結果は `docs/aipl_reply_inference_ja.pdf` にある。

目次

第 I 部	今回の変更点	2
1	変更の要約	3
2	移行が必要な点	3
2.1	文字列連結は <code>++</code> に書き換える	3
2.2	公開するアクターのメソッドに注釈を付ける	3
2.3	曖昧な式に注釈を付ける	4
2.4	<code>int</code> と <code>float</code> の混在を直す	4
2.5	二重 <code>reply</code> を直す	4
第 II 部	言語ガイド	4
3	プログラムの構成	4
4	型	5
5	式	6
6	文	6

7	アクターと通信	7
7.1	三つの送信	7
7.2	reply	7
7.3	select	7
8	型検査が見るもの	8
9	外部公開	8
10	組込み関数 (抜粋)	9
第 III 部 サンプルプログラム		9
11	g1: クラス・フィールド・send・文字列連結	9
12	g2: now / future / await と注釈	10
13	g3: 複数アクターと、アクターを跨いだ now	10
14	g4: select と reply の帰属	11
15	g5: 外部公開と注釈の必須化	12
16	負例の一覧	13
第 IV 部 ビルドと実行		13
17	処理系のビルド	13
18	プログラムの実行	13
19	型検査だけを走らせる	14
20	型と実行値の差分テスト	14
21	環境変数	14
22	関連文書	14
23	現状の限界	15

第 I 部

今回の変更点

1 変更の要約

	内容	影響
戻り値型の推論	<code>reply(v)</code> から各メソッドの戻り値型を推論する。 <code>now / future</code> の結果が <code>any</code> でなくなった	追加のみ
戻り値型注釈 被覆検査	<code>method m(x) : T</code> を書けるようになった（省略可） <code>unit</code> 以外を宣言したメソッドは全実行パスで <code>reply</code> しなければならない	追加のみ 注釈時のみ
<code>reply</code> の線形性	<code>reply</code> は高々一度。二重 <code>reply</code> は型エラー	要確認
<code>select</code> の照合	<code>case m(...)</code> を <code>m</code> の署名に照合する（arity と型）	要確認
<code>select</code> の <code>reply</code>	<code>case</code> 本体の <code>reply</code> は選択されたメッセージへの返信として型付けされる	意味の明確化
公開時の注釈必須	<code>web_expose</code> したアクターのメソッドは注釈が必須	要移行
文字列連結	<code>+</code> から <code>++</code> へ分離。 <code>+</code> は数値専用	要移行
整数演算	<code>7 + 2</code> が <code>9</code> 。ではなく <code>9</code> を返す。 <code>/</code> は <code>float</code>	値が変わる
曖昧な overload	エラーに格上げ（従来は警告して既定候補を選択）	要移行
仮引数の結線	本体の仮引数がシグネチャの引数型と繋がった。引数の型検査が実際に効くようになった	要確認

2 移行が必要な点

2.1 文字列連結は ++ に書き換える

`+` は数値専用になった。文字列を混ぜると型エラーになる。

```
print("count = " + n);    // 変更前
print("count = " ++ n);   // 変更後
```

`++` は両辺を文字列化して連結するので、どちらが文字列でなくてもよい。結合は `+` より弱いので、`"x=" ++ a + b` は `"x=" ++ (a + b)` と読まれる。

機械的に移行するには、型検査器のエラー位置に従うのが安全である。`no overload of + matches` が出た（line, col）は演算子トークンそのものを指すので、その 1 文字を `++` にして再検査すればよい。ただし列はバイトオフセットなので、日本語を含む行ではバイト単位で置換する必要がある。

2.2 公開するアクターのメソッドに注釈を付ける

`web_expose` で名指したアクターのメソッド（`init` を除く）は戻り値型の注釈が必須になった。相手からは本体が見えないので、返信の型は宣言でしか伝わらないためである。

```
method say(msg) : string { reply("echo: " ++ msg); }
```

`web_listen` だけを呼んでいて `web_expose` していないアクターは警告にとどまる（`POST /api/send` でアクター名を指定すれば到達できるが、公開の意図があるとは限らないため）。`AIOS_LAX_EXPOSE=1` でエラーを警告に落とせる。

2.3 曖昧な式に注釈を付ける

両辺が未束縛の $a + b$ は、`int` と `float` のどちらとも解釈できて `principal type` を持たない。従来は警告して `float` を選んでいたが、それは静的な型と実行結果が食い違う原因になっていたのでエラーにした。

```
method add(a, b) { reply(a + b); }           // エラー: ambiguous overload
method add(a, b) : int { reply(a + b); }      // 注釈で確定する
```

宣言した型は**期待型**として式の推論へ流れるので、注釈を書くだけでオーバーロードが一意に決まる。
`AIOS_LAX_OVERLOAD=1` で従来の警告動作に戻せる。

2.4 `int` と `float` の混在を直す

仮引数の結線により、本体が要求する型と呼び出し側が渡す型が照合されるようになった。そのため次のような不整合が表に出る。

```
class Hello {
  float count = 0.;
  method init(n) { count = n; }  // 本体は n : float を要求する
}
var h = new Hello(5);           // int を渡している -> 型エラー
var h = new Hello(5.);          // 正しい
```

2.5 二重 `reply` を直す

`reply` は高々一度でなければならない。2 度目の `reply` は待ち手がすでに 1 度目の値を受け取ったあとに来るので黙って捨てられていた。

```
method m(k) : int { reply(k); reply(k + 1); }  // 型エラー
method m(k) : int { if (k > 0) { reply(1); } else { reply(2); } } // OK
```

ループ本体の `reply` も「2 回以上ありうる」と判定される。

第 II 部

言語ガイド

3 プログラムの構成

プログラムはクラス宣言とグローバル文の並びである。

```
class Counter {
  var n = 0;           // フィールド
  method init(start) { // new のときに自動で呼ばれる
    n = start;
  }
}
```

```

method bump() : int {
    n = n + 1;
    reply(n);
}
}

var c = new Counter(10);          // グローバル文
print(now c.bump());

```

フィールドは `var x = e`; または `float x = e`; で宣言する。メソッドは `method m(p1, ..., pn) { ... }`、戻り値型を書く場合は `method m(p1, ..., pn) : T { ... }` である。

`init` という名前のメソッドは `new C(args)` のときに自動で呼ばれる。引数の個数が合わないとな生成が飛ばされる。

4 型

型	説明
<code>int, float, string, bool, unit</code>	基底型
<code>T[]</code>	配列
<code>future T</code>	<code>future</code> 送信の結果。 <code>await</code> で <code>T</code> を取り出す
<code>actor(C)</code>	クラス <code>C</code> のアクター。 <code>new C(...)</code> の型
<code>any</code>	静的に内容を保証しない境界 (<code>sender</code> 、リモート送信先)

戻り値型注釈に書けるのは `int / float / string / bool / unit / any` と、大文字で始まるクラス名 (アクター型) である。

メソッドの引数は**単相**である。シグネチャの引数型変数は全呼び出し地点で共有されるので、同じメソッドを `int` と `string` で呼ぶと衝突する。

5 式

式	意味
123, 1.5, "abc"	リテラル
x	変数・フィールド・仮引数
a ++ b	文字列連結（両辺を文字列化）。結合は最も弱い
a + b, a - b, a * b	数値演算。int 同士は int、float 同士は float
a / b	除算。int 同士でも float（整数除算はしない）
a > b, a < b, a >= b, a <= b	比較。結果は bool
new C(args)	アクターの生成
now t.m(args)	同期送信。返信が来るまで待ち、その値になる
future t.m(args)	非同期送信。future T を返す
await e	future の解決を待つ
f(args)	組込み関数の呼び出し
(e)	括弧

送信先 t は変数名か remote("host:port", "actorName") である。

既知の欠落：== と != は字句解析器と評価器にはあるが、**式の文法規則が無い**ため書けない。等値比較が必要な場合は現状 >= と <= を組み合わせるしかない。

6 文

文	意味
var x = e;	変数宣言
var x = new C(args);	アクター生成つき宣言
x = e;	代入
f(args); call f(args);	組込み関数の呼び出し
send t.m(args);	非同期送信。返信を待たない
send self.m(args);	自分自身への非同期送信
send sender.m(args);	送信元への非同期送信
send! t.m(args);	検査を緩めた送信
if (e) s if (e) s else s	条件分岐
while e do s	繰り返し（括弧は不要）
{ s ... }	ブロック
become C(args);	振る舞いの置換
select { ... }	メッセージの選択受信

now self.m() と now sender.m() は**文法上書けない**。自分への now は必ず自己デッドロックになるので、書けないことがそのまま安全側に働いている。

7 アクターと通信

7.1 三つの送信

	待つか	式か文か
<code>send a.m(x);</code>	待たない	文
<code>now a.m(x)</code>	返信まで待つ	式（結果は宣言／推論された型）
<code>future a.m(x)</code>	待たない	式（ <code>future T</code> 。 <code>await</code> で取る）

7.2 reply

メソッドは値を「返す」のではなく `reply(v)` で呼び出し側の `future` を解決する。型システムはメソッドごとに戻り値型を持ち、本体のすべての `reply` と、すべての `now` / `future` 送信地点がこの型を共有する。

- `reply` は高々一度。
- 戻り値型を `unit` 以外で宣言した場合は、全実行パスでちょうど一度になる。
- `reply` がまったく無いメソッドは戻り値 `unit` と扱われる。`send` で呼ぶ「通知」型のメソッドがこれにあたる。

7.3 select

```
select {  
  case job(k) -> { reply("done:" ++ k); }  
  timeout 500 -> { print("timed out"); }  
}
```

`case m(x1, ..., xn)` は同名メソッドの署名に照合される。引数の個数と型が合わないと型エラーになる。

`case` 本体の `reply` は、囲むメソッドではなく「選択されたメッセージ」への返信である。処理系が `case` 実行の前に返信先を選択されたメッセージへ切り替えるため、上の例の `reply` は `job` への返信になる。したがって `select` を含むメソッド自身は `unit` でよい。一方 `timeout` 本体は囲むメソッドの返信先のまま実行される。

`select` はまだ処理されていないメッセージを mailbox から探す。通常のメソッド `dispatch` が先に同名メッセージを取ってしまうと `case` には残らないので、`select` で待つ名前は直接呼ばせない設計にするのが安全である（第 III 部 g4 の実行結果を参照）。

8 型検査が見るもの

検査	内容
メソッドの存在	ローカル送信先に、その名前のメソッドがあるか
引数の個数と型	呼び出し側の実引数と、 本体が要求する型 が一致するか
reply の型	すべての reply が同じ（宣言された）型か
reply の回数	高々一度か。注釈があれば全パスでちょうど一度か
select の照合	case の arity と引数型が署名と一致するか
公開時の注釈	web_expose したアクターのメソッドに注釈があるか
オーバーロード	候補が一意に決まるか（曖昧ならエラー）

代表的なエラーメッセージを挙げる。

```
no method foo in actor(C)
type mismatch                                <- 引数や代入の型が合わない
no overload of + matches (string, int)       <- + は数値専用
ambiguous overload of + for ('a', 'a'): candidate return types are float / int
reply type mismatch: this method is declared to reply with int,
  but replies with string here
method m is declared to reply with int, but some execution path does not reply
method m may reply more than once on some path
select: case m binds 1 variable(s) but method m takes 2
actor C is reachable from outside this program, but m has no return type
annotation; ...
```

9 外部公開

web_listen(port) で HTTP ゲートウェイが起動し、web_expose(path, actorName) で公開エンドポイントに別名が付く。

エンドポイント	内容
GET /	簡易 UI
POST /api/send	to=<actor>&method=<m>&args=<a,b> で任意のアクターへ
POST /api/x/<key>	web_expose した別名へ

POST /api/send は任意のアクターに名前でも到達できる。すなわち境界を開いているのは web_listen であって、web_expose は別名を付けるだけである。型検査はこれを踏まえ、名指しされたアクターはエラー、それ以外は警告としている。

HTTP から渡される引数は文字列を int → float → string の順に解釈して作られる。境界が動的であること自体は注釈では消せないが、**どの型を約束しているか**は宣言できる。

10 組込み関数（抜粋）

分類	名前
基本	print, typeof, wait, reply, spawn
数学	sin, cos, tan, asin, acos, atan, sqrt, exp, log10, abs, floor, ceil, round
配列	array_empty, array_push, array_get, array_set, array_len
Web	web_listen, web_expose
描画 (SDL)	sdl_init, sdl_line, sdl_erase_line, sdl_clear, sdl_present
AI 呼び出し	ai_call, ai_call_with_system, model_generate, gemini_generate, openai_generate
AIOS 内省	capabilities, aios_kernel, aios_actors, aios_actor_info, aios_mailbox_len
AIOS 通信	aios_now, aios_future, aios_emit, aios_events
記憶・タスク	aios_memory_put, aios_memory_get, aios_task_create, aios_task_set, aios_tasks
プロトコル	protocol_define, protocol_start, protocol_use, protocol_state, protocol_end
リモート査読	remote_review, remote_review_ja, remote_reviewer_host

第 III 部

サンプルプログラム

本節のサンプルは docs/samples/guide/ にある。すべて型検査を通り、実行結果も掲載のとおりである。

11 g1: クラス・フィールド・send・文字列連結

```
// g1: クラス・フィールド・init・メソッド・send・文字列連結
class Greeter {
  var count = 0;                // フィールド。int で初期化

  method init(name) {           // new のときに自動で呼ばれる
    print("hello, " ++ name); // ++ は文字列連結（両辺を文字列化する）
  }

  method tick() : unit {        // 戻り値注釈。reply しないので unit
    count = count + 1;          // + は数値専用
    print("tick " ++ count);
  }
}

var g = new Greeter("AIPL");
send g.tick();                  // 非同期。返信を待たない
send g.tick();
```

```
hello, AIPL
tick 1
tick 2
```

`init` は `new Greeter("AIPL")` のときに自動で呼ばれる。`tick` は `reply` しないので `unit` を宣言してあり、被覆検査は課されない。`send` は返信を待たないので、2 つの `tick` は順に処理される。`count + 1` は `int` 同士なので `int` のままで、表示も 1, 2 と整数になる。

12 g2: now / future / await と注釈

```
// g2: now / future / await と戻り値型注釈
class Calc {
  // 注釈があると reply はこの型に照合され、全パスで reply することも検査される
  method twice(a) : int {
    reply(a * 2);
  }

  // 注釈を省くと reply から推論される (-> string)
  method label(a) {
    reply("v=" ++ a);
  }
}

var c = new Calc();

var s = now c.twice(21);      // now は式。結果は int
print("twice = " ++ (s + 1)); // int として算術に使える

var f = future c.twice(10);   // future は future int
var v = await f;              // await で取り出す
print("awaited = " ++ v);

print(now c.label(7));        // 推論された string
```

```
twice = 43
awaited = 20
v=7
```

`twice` は `int` を宣言しているので `now c.twice(21)` は `int` になり、`s + 1` という算術に使える。これは改修前にはできなかった (`now` の結果が `any` だったため `no overload of + matches (any, int)` で落ちていた)。

`label` は注釈を省いているが、`reply("v=" ++ a)` から戻り値 `string` が推論される。

13 g3: 複数アクターと、アクターを跨いだ now

```
// g3: 複数アクターと、アクターを跨いだ now
class Stock {
  var n = 10;

  method take(k) : int {
    n = n - k;
```

```

    if (n < 0) { n = 0; }
    reply(n);
  }
}

class Front {
  // Stock.take の戻り値 int が now の型になり、それが place の string へ繋がる
  method place(k) : string {
    var left = now stock.take(k);
    if (left > 0) {
      reply("ok, left=" ++ left);
    } else {
      reply("sold out");
    }
  }
}

var stock = new Stock();
var front = new Front();

print(now front.place(3));
print(now front.place(99));

```

```

ok, left=7
sold out

```

Stock.take が int を返し、それが Front.place の中の now の型になり、さらに place 自身の string へ繋がる。型はアクター境界を越えて伝播する。

place は string を宣言しているので、if の両方の枝で reply していることが検査される。片方を消すと some execution path does not reply になる。

なお $n = n - k$ で n が int なので k も int に決まり、呼び出し側の place(3) と整合する。ここで place("x") と書けば引数の型エラーになる（改修前は素通りしていた）。

14 g4: select と reply の帰属

```

// g4: select / case / timeout と、case 本体の reply の帰属
class Worker {
  // 外から届く job メッセージを受け取るメソッド
  method job(k) : string {
    reply("direct:" ++ k);
  }

  // select で job を待つ。case 本体の reply は job への返信になるので、
  // serve 自身は何も reply せず unit のままでよい
  method serve() : unit {
    print("waiting");
    select {
      case job(k) -> {

```

```

        print("got " ++ k);
        reply("via select:" ++ k);
    }
    timeout 500 -> {
        print("timed out");
    }
}
}
}

var w = new Worker();
send w.serve();
print(now w.job(1));

```

```

direct:1
waiting
timed out

```

case job(k) の本体にある reply は serve ではなく job への返信なので、serve は unit のままでよい。型検査器も case 本体では reply を job の戻り値型に照合する。

実行結果はこの例が抱える設計上の注意を示している。now w.job(1) が通常のメソッド dispatch で先に処理されたため、serve の select が動いたときには mailbox に job が残っておらず、timeout に落ちた。select で待つメッセージ名は、直接呼ばせない設計にするのが安全である。

15 g5: 外部公開と注釈の必須化

```

// g5: 外部公開すると戻り値型注釈が必須になる
//     web_listen で境界が開き、POST /api/send で任意のアクターに到達できる。
//     公開先の本体は相手から見えないので、返信の型は宣言でしか伝わらない。
class Echo {
  method say(msg) : string { // 注釈が無いと型エラーになる
    reply("echo: " ++ msg);
  }
  method note(msg) : unit {
    print("[note] " ++ msg);
  }
}

var echo = new Echo();

web_listen(8080);
web_expose("/echo", "echo");

print("POST /api/x/echo に method=say&args=hi を送ってみてください");

```

このプログラムは型検査を通る。say の注釈を外すと次のようになる。

```

[Type error] line 16, col 1: actor Echo is reachable from outside this
program, but say has no return type annotation; a remote caller cannot

```

```
see the body, so the reply type has to be declared (method say(...) : T)
```

`web_listen(8080)` を実行するとゲートウェイが起動して待ち受けに入るので、確認は次のようにする。

```
$ curl -s -X POST http://localhost:8080/api/x/echo \
      -d 'method=say&args=hi'
```

16 負例の一覧

`docs/samples/reply_inference/` には、**通らないことを確認するためのサンプルが揃っている**。

ファイル	何が起きるか
<code>s2_missing_reply</code>	<code>reply</code> 忘れの結果を算術に使う
<code>s3_conflict</code>	分岐ごとに異なる型を <code>reply</code>
<code>s5_overload_ambiguity</code>	<code>principal type</code> が無い <code>a + b</code>
<code>s8_annotation_conflict</code>	宣言と食い違う <code>reply</code>
<code>s9_missing_path</code>	一部のパスで <code>reply</code> しない
<code>s10_expose_unannotated</code>	公開アクターに注釈が無い
<code>s12_service_exposed</code>	同上（統合サンプルの公開版）
<code>s14_select_pattern</code>	<code>case</code> の <code>arity</code> が署名と違う
<code>s15_double_reply</code>	二重 <code>reply</code>

第 IV 部

ビルドと実行

17 処理系のビルド

```
cd src && make thread_repl
```

`dune` でのビルドは `src/lexer.ml` の二重生成で失敗するため、`src/Makefile` を使う。

18 プログラムの実行

REPL コマンドを書いたファイルを `-f` に渡す。

```
cat > run.repl <<'EOF'
load docs/samples/guide/g2_now_future.abcl
compile
EOF
./src/abclrepl_thread -q -f run.repl < /dev/null
```

`-f` に `.abcl` を直接渡すと REPL コマンドとして 1 行ずつ解釈されてクラス定義が読まれないので、`load` と `compile` を使うこと。REPL の主なコマンドは `load` / `compile` / `list` / `send` / `ast` / `pprint` / `script` / `exit` である。

19 型検査だけを走らせる

REPL は SDL に依存するので、型検査専用の小さなドライバを用意している。

```
ocamlfind ocamlc -package unix -thread -linkpkg -w -a -I src -o tc \
  src/location.cmo src/types.cmo src/ast.cmo src/typing_env.cmo \
  src/infer.cmo src/typecheck.cmo src/parser.cmo src/lexer.cmo \
  docs/samples/reply_inference/tc_main.ml

./tc docs/samples/guide/g3_actors.abcl
```

推論・宣言された戻り値型の表とクラスごとのシグネチャが表示される。

20 型と実行値の差分テスト

型検査が付けた戻り値型と、実行時に実際に `reply` された値の型を突き合わせるハネスがある。

```
python3 scripts/type_runtime_diff.py abcl/*abcl docs/samples/guide/*.abcl
```

処理系側は `AIOS_TYPE_TRACE=1` のとき `reply` のたびに `[rtype] Class#method = tag` を出す。この仕組みで、整数演算が `float` を返していた不整合が実際に見つかった。わざと食い違う負例には `// TYPE_RUNTIME_DIFF: expect-mismatch` と書いておけば既知として扱われる。

21 環境変数

変数	効果
<code>AIOS_QUIET=1</code>	型スキーマ等のデバッグ出力を抑制
<code>AIOS_LAX_OVERLOAD=1</code>	曖昧なオーバーロードをエラーではなく警告にする
<code>AIOS_LAX_EXPOSE=1</code>	公開アクターの注釈漏れをエラーではなく警告にする
<code>AIOS_TYPE_TRACE=1</code>	<code>reply</code> のたびに <code>[rtype]</code> 行を出す
<code>AIOS_MODEL_PROVIDER</code>	AI 呼び出しのプロバイダ選択
<code>ABCL_AI_PROVIDER</code>	同上 (旧名)
<code>REMOTE_REVIEWER_HOSTPORT</code>	<code>remote_review</code> の宛先

22 関連文書

- `docs/aipl_reply_inference_ja.pdf` — 今回の改修の設計根拠と測定結果。推論だけでは決まらない 6 点、既存アクター言語との比較、実装の詳細、退行試験の数値。
- `docs/ABCLCP_TYPE_SOUNDNESS_REPORT.md` — 型健全性の整理。
- `docs/samples/reply_inference/README.md` — 負例サンプルの一覧と意図。
- `docs/AIOS_LANGUAGE_DESIGN.md`, `docs/CAPABILITIES.md` — `AIOS` としての設計。

23 現状の限界

- `==` と `!=` が式として書けない（文法規則が無い）。
- `sender` は `any` で、`send sender.m(...)` はメソッドの存在も型も検査されない。
- リモート送信先 (`remote(...)`) の戻り値は `any` のまま。対向ノードのインタフェース記述を読み込む仕組みが無い。
- アクター型どちらも単一化で区別されない (`actor(A)` と `actor(B)` が単一化に成功する)。
- `become C(...)` はコンストラクタの引数しか検査せず、置換後のインタフェースが元と一致するかを見ていない。
- メソッドの引数は単相で、多相なメソッドは書けない。
- `int` と `float` の間に暗黙変換が無く、両辺が未束縛の `a + b` は注釈が必要。